Feature Selection & Dimensionality Reduction

A Beginner's Field Guide for Machine Learning

This guide teaches you how to choose the **right** input features for a machine-learning model — and how to shrink wide datasets down without throwing away what matters. You'll meet the three families of feature-selection methods, work through the exact tools used in the course notebooks (variance threshold, F-statistic, mutual information, Ridge, chi-squared, sequential selection, and RFE), and finish with dimensionality reduction via PCA. Every idea is explained in plain language, with runnable code and pictures. No heavy math required.

> **KEY CONCEPT — Why this matters**
> More features is not always better. Irrelevant and redundant features add noise, slow training, hurt interpretability, and make models **overfit** — performing well in training but poorly on new data. Feature selection and dimensionality reduction are how you keep the signal and drop the noise.

1. The big picture

A **feature** is one input column (also called a predictor, attribute, or X variable). The **target** is what you're trying to predict (the Y variable). **Dimensionality** is simply the number of features. As that number grows, data gets sparse and models get harder to train well — a cluster of problems called the **curse of dimensionality**.

There are two broad ways to reduce dimensionality. **Feature selection** keeps a subset of your original columns and throws the rest away. **Feature extraction** builds brand-new, fewer columns out of combinations of the originals (PCA is the classic example). Selection keeps interpretability; extraction can compress harder but the new features are less readable.

The three families of feature selection

Almost every selection method is one of three types. This map is worth memorizing — the rest of the guide fills in each box.

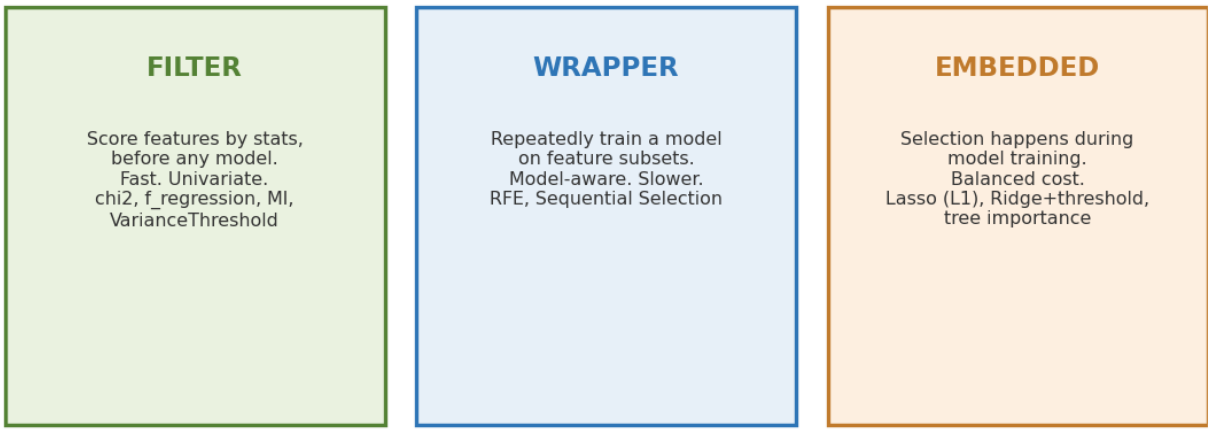

Figure 1. The three families. Filter = fast statistics before modeling; Wrapper = train a model on subsets; Embedded = selection built into training.

Family	How it works	Speed	Course tools
Filter	Score each feature with a statistic, before any model.	Fast	VarianceThreshold, f_regression, chi2, mutual information
Wrapper	Repeatedly train a model on different feature subsets and compare.	Slow	RFE, Sequential Feature Selection
Embedded	Selection happens during model training itself.	Medium	Lasso (L1), Ridge + threshold, tree importance

2. First, check for multicollinearity

Before selecting anything, it helps to spot features that **overlap**. **Multicollinearity** is when one feature is largely explained by others — they carry the same information. Classic example: *height in inches* and *height in centimeters*, which are an exact multiple of each other. That extreme case is **perfect** collinearity: an ordinary linear model's coefficients aren't even uniquely defined. More commonly features overlap partly, and coefficients become unstable and hard to trust.

The standard diagnostic is the **Variance Inflation Factor (VIF)**. For each feature, VIF tries to predict it from all the others; if they explain it well, its VIF is high and its coefficient estimate is imprecise.

```
from statsmodels.stats.outliers_influence import \
     variance_inflation_factor
import pandas as pd

vif = pd.DataFrame({
    "feature": X.columns,
    "VIF": [variance_inflation_factor(X.values, i)
            for i in range(X.shape[1])]
})
```

VIF value	Meaning	Action
around 1	Independent of the others.	Nothing to do.
5 to 10	Moderate overlap.	Worth a look.
above 10	Strong overlap.	Consider dropping/combining, or use Ridge.

These cutoffs are informal rules of thumb, not hard rules. A correlation heatmap is a quick screen too, but VIF is stronger because it catches overlap spread across *several* features at once, which pairwise correlation misses. Keep in mind VIF is mainly a diagnostic for *interpreting* linear-model coefficients — high VIF doesn't necessarily hurt out-of-sample prediction, especially for regularized or tree-based models.

3. Filter methods (fast, model-free)

Filter methods score features using statistics alone, before any model is trained. They're fast and great for a first pass, but because they look at each feature *in isolation* (**univariate**), they can miss features that only matter in combination.

3.1 Variance threshold

The simplest filter: drop features that barely change across rows. A feature with almost no **variance** carries almost no information. You set a cutoff and everything below it is removed.

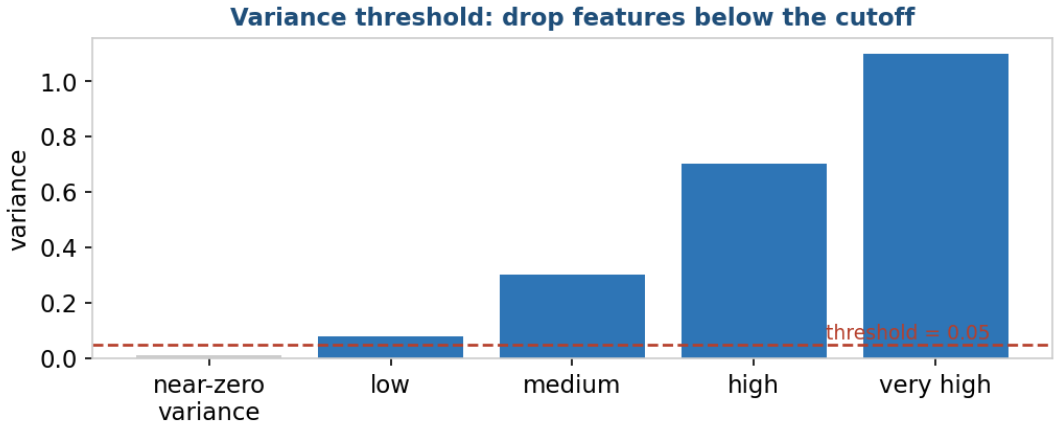

Figure 2. Variance threshold keeps only features that vary enough. The greyed bar falls below the 0.05 cutoff and is dropped.

```
from sklearn.feature_selection import VarianceThreshold

selector = VarianceThreshold(threshold=0.05)
X_train_sel = selector.fit_transform(X_train)
X_test_sel  = selector.transform(X_test)
```

KEY CONCEPT — Watch the order

Variance threshold is **unit-sensitive** — rescaling a feature changes its variance. Apply it *before* standardization, because standardizing forces every feature to variance 1 and would erase the differences. Compute the threshold on **training folds only** (it's a fitted step, so it can leak). And a near-constant feature can still matter in rare-event or anomaly-detection settings, so treat this as a rough first cut, not a final verdict.

3.2 The F-statistic (f_regression / f_classif)

The **F-statistic** scores how strongly a single feature relates to the target — but the two scikit-learn versions ask slightly different questions. `f_regression` (continuous target) measures the *linear* association between the feature and the target. `f_classif` (classification) is an **ANOVA** F-test that measures how much a feature's *mean differs across the classes* — it flags class separation, not a straight-line fit. Higher F-score (and smaller p-value) means a stronger signal either way. Both pair naturally with **SelectKBest**, which keeps the k highest-scoring features.

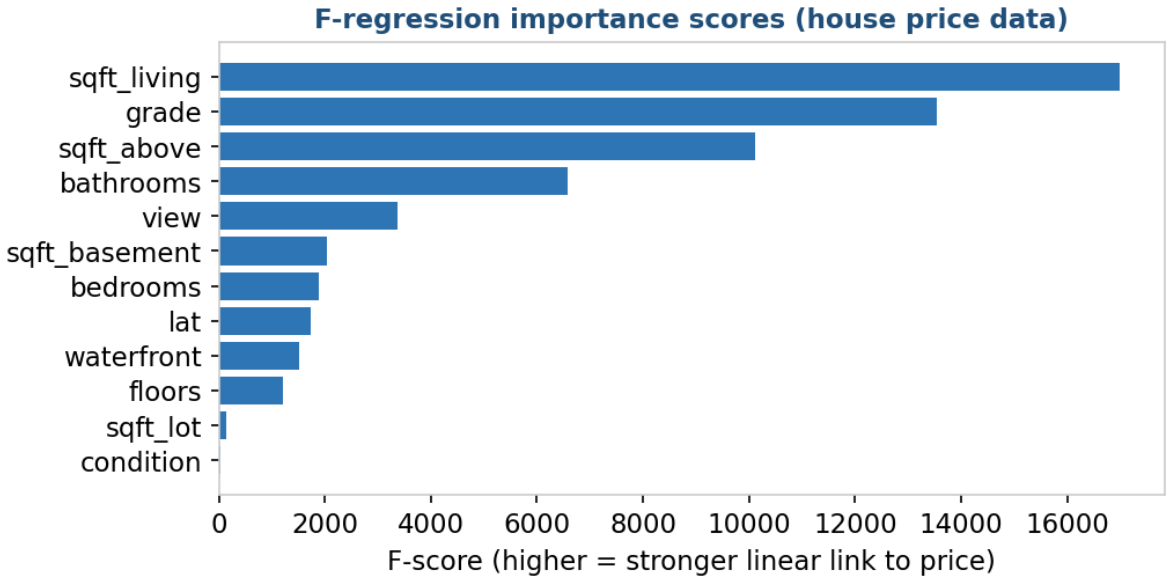

Figure 3. Real F-regression scores on the course house-price dataset. Longer bars are more strongly linked to price and are the ones SelectKBest keeps.

```
from sklearn.feature_selection import SelectKBest, f_regression

# keep the 8 best features by linear relationship to the target
selector = SelectKBest(score_func=f_regression, k=8)
selector.fit(X_train, y_train)

selected = X_train.columns[selector.get_support()]
print('Selected features:', list(selected))
```

The **support mask** from `get_support()` is a True/False array telling you which columns survived — handy for mapping back to feature names.

3.3 Mutual information (catches non-linear links)

The F-statistic only sees straight-line relationships. **Mutual information (MI)** asks a looser question: *if I know this feature, how much does my uncertainty about the target drop?* That lets it detect **non-linear** patterns the F-test misses entirely. This is MI's main advantage.

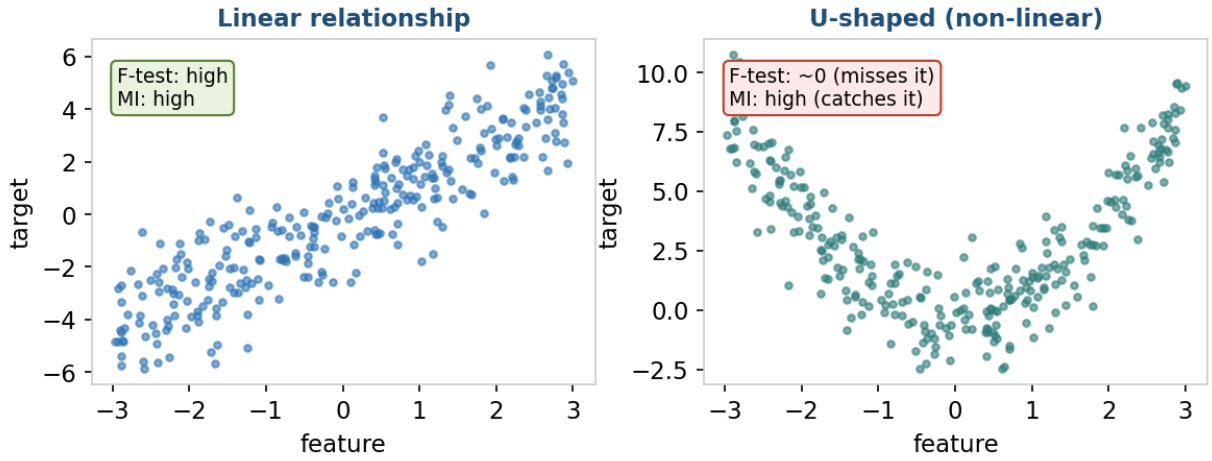

Figure 4. Left: a straight-line link both methods catch. Right: a U-shape. Because the curve goes down then up, its linear correlation is near zero, so the F-test scores it ~0 and would wrongly discard it — but mutual information still detects the dependence.

KEY CONCEPT — F-test vs. mutual information

Use the **F-test** when you expect linear relationships and want speed. Use **mutual information** when relationships may be curved or complex. MI returns importance scores, not p-values. Two cautions: MI estimates get **noisy** with limited data, and because it's still univariate it can miss a feature that only matters *jointly* with others. In theory zero MI means independence; in a real finite sample a near-zero score is not proof.

```
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import mutual_info_regression

selector = SelectKBest(score_func=mutual_info_regression, k=8)
selector.fit(X_train, y_train)
selected = X_train.columns[selector.get_support()]
print('Selected features:', list(selected))
```

3.4 Chi-squared (for classification)

For classification with **non-negative** features — especially category counts, frequencies (like bag-of-words), or one-hot-encoded categories — the **chi-squared (chi2)** test checks whether a feature and the target are **independent**. A large chi2 score and small p-value mean they're related, so the feature is worth keeping. The **null hypothesis** is that feature and target are independent; a small p-value is evidence against it.

KEY CONCEPT — Chi-squared requirements

In scikit-learn, `chi2` needs **non-negative** values. It's most natural for counts, frequencies, or categorical indicators. Continuous features can be passed if non-negative, but usually need discretizing (binning) first — and note that arbitrary binning can discard information.

```
from sklearn.feature_selection import SelectPercentile, chi2

# keep the top 30% of categorical features by chi2 score
selector = SelectPercentile(score_func=chi2, percentile=30)
selector.fit(X_cat, y_train)
selected = X_cat.columns[selector.get_support()]
print('Selected:', selected.tolist())
```

4. Embedded methods (selection during training)

Embedded methods let the model itself decide what matters, as it trains. The two you'll use most are the regularized linear models: **Lasso (L1)** and **Ridge (L2)**.

	L1 (Lasso)	L2 (Ridge)
Penalizes	Sum of absolute coefficients	Sum of squared coefficients
Effect	Can push coefficients to exactly zero	Shrinks toward zero, rarely to zero
Selects features?	Yes — zeroed features are dropped	No — needs a threshold step
Best for	Automatic feature selection	Handling multicollinearity

Ridge shines when predictors are correlated: it shares weight smoothly among them and keeps coefficients stable. It doesn't drop features on its own — the selection is a separate **model-based post-hoc** step where you rank features by scaled coefficient size and keep the strongest with **SelectFromModel**. Because that threshold is an extra choice, it should be tuned and validated, not eyeballed.

```
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV
from sklearn.feature_selection import SelectFromModel

# scaler + RidgeCV in one pipeline: scaling is fit on
# training folds only, so no leakage. RidgeCV picks alpha.
ridge = make_pipeline(
    StandardScaler(),
    RidgeCV(alphas=np.logspace(-6, 6, num=13)),
).fit(X_train, y_train)

# keep the strongest features by coefficient magnitude
rcv = ridge.named_steps['ridgecv']
sfm = SelectFromModel(rcv, threshold='median', prefit=True)
selected = feature_names[sfm.get_support()]
print('Ridge-selected features:', list(selected))
```

KEY CONCEPT — Always scale before L1 / L2

These penalties act on coefficient *size*, so features on different scales are penalized unfairly. Standardize first — and put the scaler and model in a **Pipeline** so scaling is fit only on each training fold, preventing **data leakage**. If you also tune the `SelectFromModel` threshold, do it in an outer CV loop (nested cross-validation); picking it from the same scores you report makes performance look better than it is.

5. Wrapper methods (let a model choose)

Wrapper methods repeatedly train your actual model on different feature subsets and keep whatever performs best. They're **multivariate** — they evaluate features in combination rather than in isolation. They can exploit **interactions**, but only when the underlying estimator can represent them: a tree model can discover interactions on its own, while a plain `LinearRegression` only will if you supply interaction terms. Wrappers also cost more compute than filters.

5.1 Sequential Feature Selection (SFS)

SFS builds the feature set step by step. **Forward** selection starts empty and adds the feature that most improves cross-validated performance, one at a time. **Backward** selection starts with all features and removes the least useful. It stops at your target count.

```
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.linear_model import LinearRegression

# set scoring to match your task; default is R2 for regressors
sfs = SequentialFeatureSelector(
    LinearRegression(),
    n_features_to_select=10,
    direction='forward',
    scoring='neg_root_mean_squared_error',
    cv=5,
).fit(X_train, y_train)

selected = feature_names[sfs.get_support()]
print('Forward-selected:', list(selected))
```

Set `scoring` to a metric that matches your goal — the default is estimator-dependent (R^2 for regressors, accuracy for classifiers). Note plain linear regression doesn't *need* scaling for its ranking here; scaling is essential for the L1/L2 penalties in the previous section, not for this.

5.2 Recursive Feature Elimination (RFE)

RFE starts with all features, trains the model, ranks features by importance (`coef_` or `feature_importances_`), removes the weakest, and **refits** — repeating until the target count remains. Refitting each round is the key: when a feature is removed, the importance of the survivors can shift, so RFE keeps re-judging on the survivors rather than trusting one initial ranking. Because RFE is model-aware, whether it captures non-linearities or interactions depends entirely on the estimator you give it. Your `X` should already be numeric and suitably preprocessed.

```
from sklearn.feature_selection import RFECV
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# RFECV lets cross-validation choose how many features to keep
model = make_pipeline(
    StandardScaler(),
    RFECV(LogisticRegression(max_iter=1000),
          scoring='roc_auc', cv=5),
).fit(X, y)

rfecv = model.named_steps['rfecv']
print(rfecv.support_)      # True/False: which features kept
print(rfecv.n_features_)   # optimal count CV chose
```

KEY CONCEPT — RFE trade-offs

RFE re-fits the model many times (expensive), needs an estimator that exposes `coef_` or `feature_importances_`, and is **greedy** — it makes the locally best cut each round, not a guaranteed-global-best subset. Plain RFE also doesn't validate *how many* features to keep, which is exactly why **RFECV** (shown above) is the safer default — it uses cross-validation to choose the count.

6. Dimensionality reduction with PCA

Feature selection keeps original columns. **Feature extraction** instead builds new, fewer columns. The most common is **Principal Component Analysis (PCA)** — an unsupervised method that finds new axes (**principal components**) pointing in the directions of greatest **variance** in the data. Each component is a weighted blend of the originals. The components are **orthogonal** directions, and their scores are uncorrelated in the data PCA was fit on (uncorrelated is not the same as fully independent).

You keep just the first few components — enough to retain most of the *input variance*. How many? Plot the cumulative **explained variance** and look for the **elbow**, or keep enough to reach a threshold like 90% or 95%. One important caveat: PCA ignores the target, so explained variance is not the same as *predictive* information — PCA can discard a low-variance direction that happens to be highly predictive. Always confirm with cross-validation that the reduced representation preserves task performance.

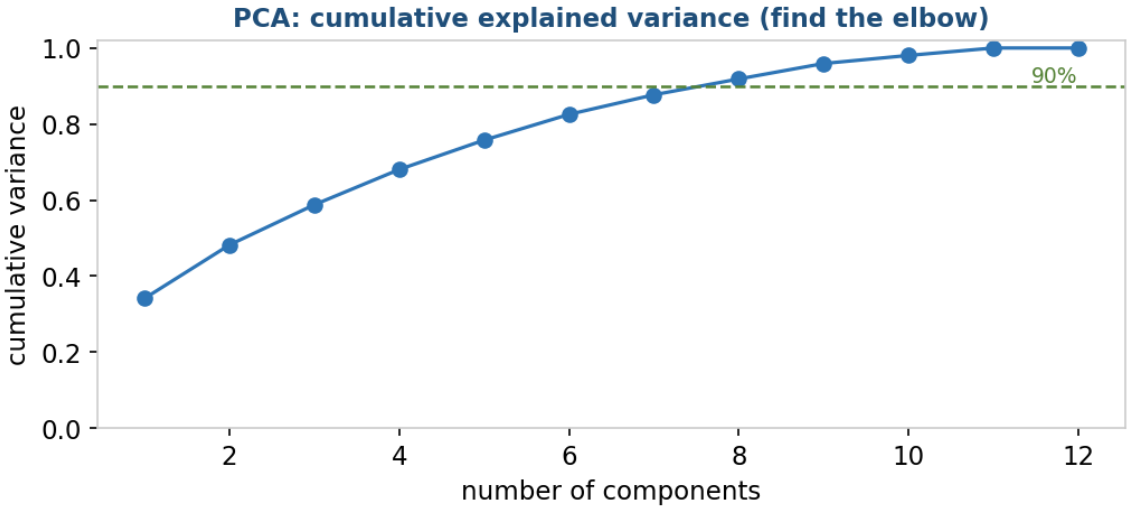

Figure 5. PCA on the course dataset. Each point adds one component; the curve shows how much total variance is captured. Here ~5-6 components already reach 90%, so the rest can be dropped.

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline

# standardize first, then keep enough components for 90% variance
pca = make_pipeline(StandardScaler(), PCA(n_components=0.90))
X_reduced = pca.fit_transform(X)
```

KEY CONCEPT — PCA cautions

PCA is sensitive to scale: **standardize first** when features have different units or incomparable ranges, so a large-unit feature doesn't dominate. (Keep raw scaling only when relative variances are meaningful by design.) Thresholds like 90-95% are common heuristics, not universal targets. The trade-off: components are **harder to interpret** than original features. PCA is unsupervised; **LDA** is the supervised cousin that maximizes class separation. For *visualizing* complex data in 2-D, **t-SNE** is popular but is a visualization tool, not a general preprocessing step.

7. Putting it together: a safe workflow

A reliable feature-selection process looks like this. The golden rule throughout: fit every learned step (scalers, selectors, PCA) on **training data only**, inside a cross-validation pipeline, so no information leaks from validation or test data.

Step	What to do	Tools
1. Baseline	Fit a model on all features first, as a reference point.	Baseline model + CV
2. Clean overlap	Check multicollinearity; drop or combine redundant features.	VIF, correlation heatmap
3. Quick filter	Cheaply drop weak features.	VarianceThreshold, f_/chi2, MI
4. Refine	Let a model choose the subset that predicts best.	RFE, SFS, Lasso, Ridge+SelectFromModel
5. Validate	Confirm the smaller set holds up on unseen data.	Cross-validation, held-out test set

KEY CONCEPT — Selection can overfit too

If you test many feature subsets and keep whatever scored best, you can **overfit the selection itself**. Always judge the final subset with proper cross-validation and a clean, untouched test set — and let **domain knowledge** sanity-check what the scores suggest.

8. Vocabulary & concepts

A quick-reference glossary of every key term in this guide. Keep it beside you while working through the notebooks.

Term	Definition
Adjusted R-squared	R-squared adjusted to penalize adding features that don't help; good for comparing models with different feature counts.
ANOVA	Compares variation between groups vs. within groups; the basis of f_classif. (f_regression instead derives its F from feature-target correlation.)
Autoencoder	A neural net that compresses input to a small latent space and rebuilds it — a non-linear form of feature extraction.
Baseline model	The all-features model you build first as a reference for judging any reduced set.
Chi-squared selection	A filter for classification testing dependence between a non-negative feature (counts, frequencies, encoded categories) and the class label.
Coefficient magnitude	Absolute size of a coefficient; on comparable scales, larger can mean more influence — but read cautiously when features correlate.
Correlation	Direction and strength of association; Pearson measures linear links between continuous variables.
Correlation heatmap	A colour-coded matrix of pairwise correlations; spots redundant, overlapping predictors.
Cross-validation	Training and validating across multiple data splits for a reliable out-of-sample estimate.
Curse of dimensionality	The problems that grow with feature count: sparse data, distance measures becoming less discriminative, more overfitting.
Data leakage	Accidentally using validation/test info during selection or preprocessing; fit learned steps on training data only.
Dimensionality	The number of input features per observation.
Dimensionality reduction	Representing data with fewer dimensions; includes selection (keep originals) and extraction (new features).
Domain knowledge	Subject expertise used to judge which features are meaningful; should complement automated scores.
Elbow method	Plotting explained variance or performance and picking the bend of diminishing returns.
Embedded method	Selection that happens during training, e.g. Lasso, tree importance, model-based selection.
Explained variance	Fraction of input variance a component captures; thresholds like 90-95% are common heuristics, not universal targets.
F-statistic	A ratio of explained to unexplained variation; larger suggests a stronger feature-target link.
f_classif / f_regression	f_classif is an ANOVA F-test (class-mean differences); f_regression derives an F from each feature's linear correlation with a continuous target.
Feature	An input variable (predictor, attribute, X) used to estimate the target.

Term	Definition
Feature extraction	Reducing dimensions by building new features (PCA, LDA, SVD, autoencoders); t-SNE is related but is a visualization tool, not general preprocessing.
Feature importance	A model- and method-dependent score summarizing a feature's association with predictions; coefficients, impurity, permutation and SHAP answer different questions.
Feature interaction	When one feature's effect depends on another; univariate filters can miss these.
Feature selection	Keeping relevant original features and discarding irrelevant, redundant, or noisy ones.
Filter method	Model-free scoring by statistics before training (correlation, variance, chi2, F-test, MI).
Forward / backward selection	Wrapper methods that add features from none, or remove from all, by validated performance.
Generalization	Performing well on new, unseen data rather than memorizing training noise.
High-dimensional data	Data with very many features, especially near or above the number of observations.
Irrelevant feature	A predictor with no stable useful information; adds noise and overfitting risk.
L1 regularization	Penalty on absolute coefficients; can zero some out (Lasso), giving embedded selection.
L2 regularization	Penalty on squared coefficients; shrinks and stabilizes under multicollinearity (Ridge).
Lasso regression	Linear model with L1; selects features by zeroing coefficients; unstable among correlated ones.
Latent space	A compressed learned representation, e.g. from an autoencoder.
LDA	Supervised extraction that finds directions maximizing class separation.
LIME	Local, model-agnostic explanation of a single prediction via a simple surrogate model.
Low-variance feature	A feature that barely changes; often weak, though low variance alone isn't proof.
Model-based selection	Keeping features whose learned coefficients or importances beat a threshold (SelectFromModel).
Multicollinearity	When one predictor is largely explained by others; destabilizes linear coefficients.
Mutual information	How much knowing one variable reduces uncertainty about another; detects non-linear links.
mutual_info_classif / _regression	Scikit-learn MI scorers for categorical and continuous targets; scores, not p-values.
Nominal feature	An unordered category (region, marital status); usually one-hot encoded.
Null hypothesis	The default 'no effect' claim a test checks; in chi2 selection, that feature and target are independent.
One-hot encoding	Representing each category as its own 0/1 column; avoids fake ordering but adds dimensions.
Ordinal feature	An ordered category (low/medium/high); ordinal encoding preserves the order.
Orthogonal components	Right-angle, uncorrelated directions — PCA components are orthogonal.
Out-of-sample performance	Performance on data not used to train or select; the real evidence selection helped.

Term	Definition
Overfitting	Learning training noise and failing on new data; selection helps but can itself overfit.
p-value	Assuming the null hypothesis and test model hold, the probability of a result at least this extreme; not the probability the null is true.
Permutation importance	Shuffle one feature and measure the drop in validated performance; model-agnostic.
Pipeline	A chained preprocess-select-model workflow that prevents leakage by fitting on training folds only.
Principal component	A new feature that is a weighted blend of originals; the first captures the most variance.
PCA	Unsupervised extraction onto orthogonal max-variance directions; standardize when scales differ. Retained variance is not the same as predictive information.
Random forest	A tree ensemble capturing non-linear interactions; gives impurity-based importances, but check them with permutation importance on held-out data.
RFE	Wrapper that refits a model, ranks by importance, and removes the weakest until the target count remains.
Redundant feature	A predictor whose information is already covered by others.
Regularization	A complexity penalty added to training; L1 creates sparsity, L2 shrinks smoothly.
Ridge regression	Linear model with L2; great for correlated predictors; needs a threshold to hard-select.
RidgeCV	Ridge that picks its penalty strength by cross-validation.
SelectFromModel	Meta-transformer keeping features whose coefficient/importance meets a threshold.
SelectKBest	Univariate selector keeping the k highest-scoring features.
SelectPercentile	Univariate selector keeping a top percentage of features.
Sequential Feature Selection	Wrapper adding or removing features by cross-validated performance.
Sequential floating selection	A bidirectional wrapper that alternates adds and removes to escape greedy mistakes.
SHAP	Game-theoretic feature-contribution explanations, local and global; not proof of causality.
SVD	Matrix factorization for dimensionality reduction; truncated SVD suits sparse text matrices.
Standardization	Rescaling to mean 0, sd 1; essential for fair L1/L2 penalties, and appropriate before PCA when features have different units or scales.
Stopping criterion	The rule that ends a sequential search (target count, no improvement, budget).
Support mask	A True/False array of which features a selector kept; from get_support().
Target variable	The outcome (response, label, Y) a supervised model predicts.
t-SNE	Non-linear method to visualize high-dimensional data in 2-3D; interpret distances cautiously.
Univariate selection	Scoring each feature separately; fast screening but misses interactions.

Term	Definition
Variance	How spread out a feature's values are around their mean (the square of standard deviation).
Variance Inflation Factor	Multicollinearity diagnostic 1/(1-R2); >5-10 warns of overlap; infinite = perfect dependence.
Variance threshold	Filter removing features whose variance falls below a cutoff; fast but unit-sensitive.
Wrapper method	Selection that retrains a model on candidate subsets; can exploit interactions when the estimator supports them, but is slower and can overfit.

9. Further reading & resources

Curated beginner-friendly resources to go deeper. Documentation first, then visual explainers, then hands-on code tutorials.

Official documentation & API guides

Scikit-Learn Feature Selection User Guide — Filter, wrapper, and embedded methods (SelectKBest, VarianceThreshold, RFE, SelectFromModel, Lasso).
https://scikit-learn.org/stable/modules/feature_selection.html

Scikit-Learn Decomposing Signals / PCA Guide — PCA, Incremental PCA, and truncated SVD.
https://scikit-learn.org/stable/modules/decomposition.html

Visual & intuitive explanations

StatQuest: PCA Step-by-Step — The definitive visual walkthrough of how PCA projects data into components.
https://www.youtube.com/watch?v=FgakZw6K1QQ

StatQuest: Feature Selection — Easy-to-follow explanation of how feature importance and selection work.
https://www.youtube.com/watch?v=YaKMeAlHgqQ

Kaggle Learn: Feature Engineering — Interactive micro-course covering mutual information and feature creation.
https://www.kaggle.com/learn/feature-engineering

Distill.pub: How to Use t-SNE Effectively — Interactive visual guide to reading t-SNE for non-linear reduction.
https://distill.pub/2016/misread-tsne/

Step-by-step code tutorials

ML Mastery: Feature Selection for ML in Python — Numerical vs. categorical feature selection, end to end.
https://machinelearningmastery.com/feature-selection-machine-learning-python/

ML Mastery: PCA for Dimensionality Reduction — Hands-on PCA pipelines and choosing n_components.
https://machinelearningmastery.com/principal-components-analysis-for-dimensionality-reduction-in-python/

ML Mastery: Dimensionality Reduction Algorithms in Python — Overview of PCA, SVD, LDA, and manifold learning (Isomap/LLE).
https://machinelearningmastery.com/dimensionality-reduction-algorithms-with-python/

Feature Selection & Dimensionality Reduction — A Beginner's Field Guide

Part of the AI Foundations Series